FAVR — Reproducibility Document

April 2026 · v0.1.0

Flexible Attack Vector Risk — Mathematically optimal vulnerability patching for real codebases

1. System Overview

FAVR is a desktop application that scans any software project directory, auto-discovers services and dependencies, queries real CVE databases (OSV.dev, NVD, GHSA, CISA KEV), then runs a full mathematical analysis pipeline to produce an optimal patching strategy. Unlike tools that sort vulnerabilities by CVSS score alone, FAVR models how compromise propagates through dependency chains and uses Monte Carlo simulation to find the patch ordering that reduces aggregate risk fastest.

1.1 Architecture

```
┌──────────────────────────────────────────────────────────┐
│                     Electron Desktop App                 │
│  React 18 UI  <----  IPC  ---->  Node.js Main Process     │
│  (Renderer)                       Codebase Analyzer       │
│  - Dashboard                          |                   │
│  - D3 Charts                      Analysis Engine         │
│  - What-If                        (10-phase pipeline)     │
│  - Reports                                                │
└──────────────────────────────────────────────────────────┘
        |                                   |
        v                                   v
  User's browser                     External APIs (free)
  (exported report)              OSV.dev, FIRST.org EPSS, NVD
                                 GitHub Advisories, CISA KEV
```

1.2 Analysis Pipeline

#	Phase	Method
1	Service Discovery	Scan manifests: package.json, go.mod, requirements.txt, Cargo.toml, pom.xml, Gemfile
2	Dependency Mapping	Infer inter-service relationships from imports and shared packages
3	Vulnerability Lookup	Query OSV.dev for known CVEs per package+version (free, no API key)
4	Multi-Source Enrichment	Cross-reference NVD, GitHub Security Advisories (GHSA), CISA KEV catalog
5	EPSS Scoring	Fetch real Exploit Prediction Scoring System probabilities from FIRST.org
6	Attack Graph	Build directed graph: services = nodes, dependencies = weighted edges
7	Bayesian Propagation	Propagate compromise probabilities through attack graph until convergence
8	Monte Carlo Simulation	Simulate N patch orderings; find sequence minimizing cumulative risk area
9	Pareto Optimization	Compute Pareto frontier over risk-reduction vs. cost vs. downtime
10	Schedule & Compliance	Assign patches to maintenance windows; flag compliance deadline violations

2. Prerequisites

Required

Optional API Keys

Tool	Version
Node.js	≥ 20.x
npm	≥ 10.x
Git	≥ 2.x
OS	macOS, Linux, or Windows

Key	Source	Purpose
OPENROUTER_API_KEY	openrouter.ai	AI-assisted remediation
GEMINI_API_KEY	Google AI Studio	Task classification

No keys are required for core scanning. OSV.dev, EPSS, and KEV are free and keyless.

3. Reproduction Steps

3.1 Clone and Install

```
git clone https://github.com/<owner>/FAVR.git
cd FAVR
npm install
```

3.2 Configure Environment (optional)

```
cp .env.example .env
# Add API keys if desired:
# OPENROUTER_API_KEY=sk-or-v1-...
# GEMINI_API_KEY=AIza...
```

> If no .env is provided, the app runs fully using the built-in demo scenario and free keyless APIs.

3.3 Run in Development Mode

```
npm run dev
```

This launches the Electron app with hot-reload via electron-vite. The app window opens automatically.

3.4 Build for Production

```
npm run build        # Compile TypeScript + bundle
npm run package      # Package as distributable (.dmg / .exe / .AppImage)
```

3.5 CLI Usage (Headless / CI)

FAVR ships a standalone CLI package for terminal and CI/CD use:

```
cd packages/favr-cli
npm run build
node dist/index.js <project-path> --format table
```

Flag	Description
-f, --format <fmt>	Output: table, json, html, sarif
-t, --threshold <sev>	Fail if any finding meets/exceeds severity or CVSS score
--diff	Compare against last scan; report only new/worsened vulns
--iterations <n>	Monte Carlo iterations (default: 500)
-o, --output <file>	Write output to file

4. User Guide

4.1 Scanning a Real Codebase (Desktop App)

1. Launch FAVR (npm run dev)
2. Click **Browse** and select a project directory containing supported package manifests
3. Click **Scan Codebase** — real-time progress shows each pipeline phase

4. Dashboard displays: system risk score, risk reduction %, vulnerability table, dependency graph, service heatmap, priority patches with CVSS-vs-EPSS divergence, and compliance alerts

4.2 Supported Ecosystems

Ecosystem	Manifest Files Detected
Node.js / npm	package.json, package-lock.json, yarn.lock
Python / pip	requirements.txt, Pipfile, pyproject.toml
Go	go.mod, go.sum
Rust	Cargo.toml, Cargo.lock
Java / Maven	pom.xml
Ruby / Bundler	Gemfile, Gemfile.lock
Docker	Dockerfile (base image scanning)

4.3 What-If Scenario Modeling

After scanning, the **What-If** view lets you explore constrained scenarios:

- **Budget cap** — "We have 20 person-hours" → see which patches fit and residual risk
- **Time horizon** — "Next 7 days only" → filter by maintenance windows
- **Toggle patches** — enable/disable individual CVEs to see impact on system risk

4.4 Report Export

Click **Export Report** to generate a self-contained HTML file with executive summary, full vulnerability table, dependency graph, heatmap, and recommended patch schedule.

4.5 Built-in Demo (Offline)

Click **Demo** to load the Meridian Financial scenario — 15 real CVEs across 5 services. Works fully offline to demonstrate all analysis capabilities without a target project.

5. Algorithm Details

5.1 Bayesian Risk Propagation

Each service *s* has a base compromise probability *P0(s)*. For each directed dependency edge *(u -> v)* with propagation weight *w*, the updated probability is:

```
P(v) = 1 - (1 - P0(v)) * PRODUCT( 1 - w(u,v) * P(u) )   for all u -> v
```

This iterates over the full graph until max change < 1e-6. The result: services downstream of compromised nodes inherit proportional risk, even if they have no direct CVEs.

5.2 Monte Carlo Optimal Patch Ordering

Given *k* open vulnerabilities, we sample *N* random permutations of patch order (default N=500). For each permutation, we simulate patching one CVE at a time, re-running Bayesian propagation after each patch, and compute the area under the risk-over-time curve (AUC). The permutation with the lowest AUC is the optimal order. Convergence is validated by comparing top-10 orderings across the final 20% of iterations.

5.3 Pareto Frontier

Each candidate patch schedule is scored on three objectives:

1. **Risk reduction** (maximize)
2. **Remediation cost** in person-hours (minimize)
3. **Downtime** in minutes (minimize)

The Pareto frontier is the set of schedules where no objective can improve without worsening another. Users select their preferred trade-off point.

5.4 EPSS vs. CVSS Divergence

FAVR highlights CVEs where EPSS (real-world exploit probability) significantly diverges from CVSS (theoretical severity). A CVE with CVSS 9.8 but EPSS 0.01 is less urgent than CVSS 6.5 with EPSS 0.85. This is a key differentiator: FAVR uses empirical exploit data, not just severity ratings.

5.5 Compliance-Aware Scheduling

CVEs in the CISA KEV catalog carry mandatory remediation deadlines. FAVR enforces these as hard constraints in the scheduling phase, ensuring compliance-critical patches are ordered before their deadlines regardless of risk score.

6. Data Sources

Source	Endpoint	Data Provided	Auth
OSV.dev	api.osv.dev/v1/query	CVE lookups by package ecosystem + version	None (free)
FIRST.org EPSS	api.first.org/data/v1/epss	Exploit probability scores (updated daily)	None (free)
NVD	services.nvd.nist.gov/rest/json/cves/2.0	Authoritative CVSS scores, CWE mappings	Optional key
GitHub Advisories	api.github.com/advisories	GHSA cross-references	Optional token
CISA KEV	cisa.gov/.../known_exploited_vulnerabilities.json	Known exploited vulnerabilities catalog	None (free)

> **Determinism note:** Because EPSS scores update daily and CVE databases grow over time, scan results may differ slightly across days. FAVR caches enrichment data locally. Re-running within the cache TTL produces identical output. The built-in Meridian Financial demo always produces deterministic results.

7. Project Structure

```
FAVR/
 src/
  main/                          # Electron main process
   engine/                       # Analysis pipeline
    index.ts                     #   Orchestrator (10-phase pipeline)
    attack-graph.ts              #   Attack graph construction
    bayesian.ts                  #   Bayesian risk propagation
    monte-carlo.ts               #   Monte Carlo simulation
    pareto.ts                    #   Pareto frontier computation
    blast-radius.ts              #   Blast radius analysis
    scheduler.ts                 #   Maintenance window scheduling
    compliance.ts                #   Compliance framework checks
    epss.ts                      #   EPSS fetch + estimation
    what-if.ts                   #   What-If scenario engine
   ingest/                       # Codebase scanning
    codebase-analyzer.ts         #   Service/dep/vuln discovery
    vuln-data-pipeline.ts        #   Multi-source CVE enrichment
    scan-history.ts              #   Scan result persistence
   ipc.ts                        # IPC bridge (UI <-> engine)
  renderer/                      # React 18 UI
   components/
    Dashboard.tsx                #   Main results view
    WhatIfView.tsx               #   Scenario modeling
    RemediationWorkspace.tsx      #   AI-assisted fix workspace
    charts/                      #   D3 + Recharts visualizations
   stores/                       # Zustand state management
  preload/index.ts               # Electron preload bridge
 packages/
  favr-core/                     # Headless engine (no Electron dep)
  favr-cli/                      # CLI for terminal / CI/CD
 electron.vite.config.ts         # Build configuration
 package.json                    # Dependencies + scripts
```

8. Tech Stack

Layer	Technology
Desktop Shell	Electron 33
Build System	electron-vite + Vite 6
UI Framework	React 18 + TypeScript 5
Styling	Tailwind CSS 3
State Management	Zustand 4
Visualizations	D3.js 7 + Recharts 3

Layer	Technology
Local Storage	electron-store
Icons	Lucide React
AI Gateway	OpenRouter (optional)
CLI Parser	Commander.js
Language	TypeScript (strict)
Package Mgmt	npm workspaces

9. Validation

9.1 Built-in Benchmark

The repository includes a Log4j benchmark dataset (src/main/data/log4j-benchmark.ts) with known CVEs and expected severity rankings.

9.2 Expected Demo Output

- 15 vulnerabilities discovered across 5 services
- System risk score > 0.6 (60%)
- Optimal patch ordering achieves > 30% greater risk reduction than naive CVSS ordering
- At least 2 CVEs flagged for CVSS-vs-EPSS divergence
- Compliance deadline alerts for PCI-DSS and HIPAA frameworks

9.3 Manual Verification Steps

1. Run `npm run dev` — app launches with no console errors
2. Click Demo — analysis completes in < 5 seconds
3. Verify all 6 chart types render: dependency graph, heatmap, severity donut, Pareto frontier, risk curves, before/after
4. Open What-If — adjust budget slider, verify residual risk updates
5. Click Export Report — opens a clean HTML file in browser
6. Point at a real project with known CVEs — verify end-to-end scan completes

10. Key Differentiators

FAVR	Traditional Tools
Scans real codebases automatically	Require manual CVE input
Models risk propagation through dependencies	Treat each CVE independently
Monte Carlo optimal patch ordering	Sort by CVSS (proven suboptimal)
Real EPSS scores from FIRST.org	Static severity ratings only
Pareto optimization (risk x cost x downtime)	Single-dimensional ranking
Maintenance-window-aware scheduling	"Patch everything now"
Compliance deadline tracking (PCI-DSS, HIPAA, SOC2)	No compliance awareness
Works offline with built-in demo	Requires cloud connectivity

— End of Document —